Index of Experiments

Exp No.	Experiment Title	Key Concepts	Status
EXP 1	**Complex Number Arithmetic & Operator Overloading** `EXP1.cpp`	Classes and Objects, Constructors, Operator Overloading (+, *, >>, <<), Friend Functions	Completed
EXP 2	**Staff Salary Calculation & Record Sorting** `EXP2.cpp`	Classes, Array of Objects, Member Functions, Sorting Algorithm, Encapsulation	Completed
EXP 3	**Publication Company (Inheritance & Exception Handling)** `EXP3.cpp`	Single Inheritance, Exception Handling (try-throw-catch), Data Validation, Polymorphism	Completed
EXP 4	**Matrix Operations (Addition, Subtraction & Multiplication)** `EXP4.cpp`	2D Arrays in Classes, Object as Function Arguments, Return by Value, Matrix Algebra	Completed
EXP 5	**Multiple Inheritance, Constructors & Destructors Order** `EXP5.cpp`	Multiple Inheritance, Constructor Chaining, Copy Constructor, Destructor Invocation Order	Completed
EXP 6	**File Input and Output Operations (ofstream & ifstream)** `EXP6.cpp`	File Streams, Persistent Storage, Error Handling with is_open(), Line-by-line reading	Completed
EXP 7	**Demonstration of All Types of Inheritance** `EXP7.cpp`	Single, Multiple, Multilevel, and Hierarchical Inheritance, Namespaces, Code Organization	Completed
EXP 8	**Generic Class Template (Library Management System)** `EXP8.cpp`	Class Templates, Generic Programming, Type Independence, Encapsulation	Completed
EXP 9	**Standard Template Library (STL Map for State Population)** `EXP9.cpp`	STL Containers, std::map, Key-Value Pairs, Iterators, find() Method	Completed
EXP 10	**ATM Machine Simulation & Custom Exception Handling** `EXP10.cpp`	Exception Hierarchy, Custom Exceptions (Arithmetic & IllegalArgument), Input Validation, Menu-driven loop	Completed

EXP 1

EXP1.cpp

Complex Number Arithmetic & Operator Overloading

Aim / Objective: Implement a C++ program to design a class 'Complex' to represent complex numbers and overload arithmetic operators '+' and '*' to perform addition and multiplication of two complex numbers, and stream operators '>>' and '<<' for I/O operations.
Core Concepts: Classes and Objects, Constructors, Operator Overloading (+, *, >>, <<), Friend Functions

SOURCE CODE:

```
#include <iostream>
using namespace std;

class Complex
{
private:
    int real;
    int imag;

public:
    // Default Constructor
    Complex()
    {
        real = 0;
        imag = 0;
    }

    // Parameterized Constructor
    Complex(int r, int i)
    {
        real = r;
        imag = i;
    }

    // Overload + operator
    Complex operator +(Complex c)
    {
        Complex temp;
        temp.real = real + c.real;
        temp.imag = imag + c.imag;
        return temp;
    }

    // Overload * operator
    Complex operator *(Complex c)
    {
        Complex temp;
        temp.real = (real * c.real) - (imag * c.imag);
        temp.imag = (real * c.imag) + (imag * c.real);
        return temp;
    }

    // Overload >> operator
    friend istream& operator >>(istream& in, Complex& c);

    // Overload << operator
    friend ostream& operator <<(ostream& out, Complex& c);
};
```

```
// Definition of >> operator
istream& operator >>(istream& in, Complex& c)
{
    cout << "Enter Real Part : ";
    in >> c.real;

    cout << "Enter Imaginary Part : ";
    in >> c.imag;

    return in;
}

// Definition of << operator
ostream& operator <<(ostream& out, Complex& c)
{
    out << c.real;

    if (c.imag ≥ 0)
        out << "+" << c.imag << "i";
    else
        out << c.imag << "i";

    return out;
}

// Main Function
int main()
{
    Complex c1, c2, sum, product;

    cout << "Enter First Complex Number" << endl;
    cin >> c1;

    cout << "\nEnter Second Complex Number" << endl;
    cin >> c2;

    sum = c1 + c2;
    product = c1 * c2;

    cout << "\nFirst Complex Number = " << c1 << endl;
    cout << "Second Complex Number = " << c2 << endl;

    cout << "\nAddition = " << sum << endl;
    cout << "Multiplication = " << product << endl;

    return 0;
}
```

TEST INPUT:

```
Enter First Complex Number: 3 4
Enter Second Complex Number: 1 2
```

EXECUTION OUTPUT:

Terminal — ./a.out

```
Enter First Complex Number
Enter Real Part : Enter Imaginary Part :
Enter Second Complex Number
Enter Real Part : Enter Imaginary Part :

First Complex Number = 3+4i
```

```
Second Complex Number = 1+2i

Addition = 4+6i
Multiplication = -5+10i
```

EXP 2

EXP2.cpp

Staff Salary Calculation & Record Sorting

Aim / Objective: Develop a C++ program to manage staff payroll records. Calculate gross and net salary based on basic pay, DA (25%), HRA (800), and Income Tax (15%). Sort staff records alphabetically by name and generate clean pay slips.
Core Concepts: Classes, Array of Objects, Member Functions, Sorting Algorithm, Encapsulation

SOURCE CODE:

```
//EXP2_Salary
#include<iostream>
#include<string>

using namespace std;


class staff
{
private:
  int Staff_ID;
  string name;
  float Basic_Salary;
  float DA, HRA, Itax, NetSalary;

public:

  void accept()
  {
    cout<<"\nEnter staff ID : \t";
    cin>> Staff_ID;
    cout<<"\nEnter staff Name : \t";
    cin>> name;
    cout<<"\nEnter staff Salary : \t";
    cin>> Basic_Salary;
  }//accept

  void calculate()
  {
    DA = Basic_Salary * 0.25 ;
    HRA = 800;
    Itax = Basic_Salary *0.15;
    NetSalary= Basic_Salary + DA + HRA -Itax;
  }//calc

  void display()
  {
    cout<<"\n\n\nStaff ID \t"<< Staff_ID;
    cout<<"\nStaff Name \t"<< name;
    cout<<"\nBasic Salary \t"<< Basic_Salary;
    cout<<"\nDA(25%) \t"<< DA;
    cout<<"\nHRA \t"<< HRA;
    cout<<"\nIncome Tax\t"<< Itax;
    cout<<"\nNet Salary \t"<< NetSalary << endl;

  }//display

  string getName()
  {
```

```
    return name;
  } //name

}; // class staff

int main()
{
  int i,j,n;

  cout<<"Enter Number of staff you want to fill\t";
  cin >> n;
  staff s[n];

  for(i=0;i<n;i++)
  {
    cout<<"\nEnter staff Details of staff "<< i +1 << endl;
    s[i].accept();
    s[i].calculate();
  }//for loop Accept data

  for(i=0;i<n-1;i++)
  {
    for(j=i+1;j<n;j++)
    {
      if(s[i].getName()>s[j].getName())
      {
        swap(s[i],s[j]);
      }
    }
  }//for loop sorting

  cout<<"\n staff pay slip";

  for(i=0;i<n;i++)
  {
    s[i].display();
  }//for loop for display

  return 0;


}//main
```

TEST INPUT:

```
Enter Number of staff: 2
Staff 1: ID = 101, Name = Alice, Basic = 50000
Staff 2: ID = 102, Name = Bob, Basic = 60000
```

EXECUTION OUTPUT:

```
Terminal — ./a.out

Enter Number of staff you want to fill
Enter staff Details of staff 1
Enter staff ID :
Enter staff Name :
Enter staff Salary :
Enter staff Details of staff 2
Enter staff ID :
Enter staff Name :
Enter staff Salary :
```

```
 staff pay slip

Staff ID        101
Staff Name      Alice
Basic Salary    50000
DA(25%)         12500
HRA     800
Income Tax      7500
Net Salary      55800

Staff ID        102
Staff Name      Bob
Basic Salary    60000
DA(25%)         15000
HRA     800
Income Tax      9000
Net Salary      66800
```

EXP 3

EXP3.cpp

Publication Company (Inheritance & Exception Handling)

Aim / Objective: Imagine a publishing company that markets both book and audio-cassette versions of its works. Create a class 'publications' with data members 'title' and 'price'. Derive two classes 'book' (pagecount) and 'disc' (playing time). Incorporate exception handling for invalid inputs (negative price, page count, or time).
Core Concepts: Single Inheritance, Exception Handling (try-throw-catch), Data Validation, Polymorphism

SOURCE CODE:

```
#include<iostream>
#include<string>
using namespace std;

class publications
{
protected:
    float price;
    string title;

public:
    void details()
    {
        try
        {
            cout << "Enter the title: ";
            cin >> ws;
            getline(cin, title);

            cout << "Enter price: ";
            cin >> price;

            if(price < 0)
                throw price;
        }
        catch( ... )
        {
            cout << "Invalid price entered!" << endl;
            setting_zero();
        }
    }

    void display()
    {
        cout << "Title : " << title << endl;
        cout << "Price : " << price << endl;
    }

    void setting_zero()
    {
        title = "Invalid";
        price = 0;
    }
};

class book : public publications
{
private:
    int pagecount;
```

```
public:
    void details_book()
    {
        details();

        try
        {
            cout << "Enter page count: ";
            cin >> pagecount;

            if(pagecount < 0)
                throw pagecount;
        }
        catch( ... )
        {
            cout << "Invalid page count entered!" << endl;
            pagecount = 0;
        }
    }

    void display_book()
    {
        cout << "\n********** BOOK DETAILS **********" << endl;
        display();
        cout << "Page Count : " << pagecount << endl;
    }
};

class disc : public publications
{
private:
    int time;

public:
    void details_disc()
    {
        details();

        try
        {
            cout << "Enter playing time (minutes): ";
            cin >> time;

            if(time < 0)
                throw time;
        }
        catch( ... )
        {
            cout << "Invalid playing time entered!" << endl;
            time = 0;
        }
    }

    void display_disc()
    {
        cout << "\n********** DISC DETAILS **********" << endl;
        display();
        cout << "Playing Time : " << time << " minutes" << endl;
    }
};

int main()
{
    book b;
    disc d;

    cout << "Enter Book Details" << endl;
```

```
    b.details_book();

    cout << "\nEnter Disc Details" << endl;
    d.details_disc();

    cout << "\nDisplaying Details\n";

    b.display_book();
    d.display_disc();

    return 0;
}
```

TEST INPUT:

```
Book Details: Title = C++ Primer, Price = 450.50, Pages = 800
Disc Details: Title = Mastering Audio, Price = 299.99, Time = 60 min
```

EXECUTION OUTPUT:

```
Terminal — ./a.out

Enter Book Details
Enter the title: Enter price: Enter page count:

Enter Disc Details
Enter the title: Enter price: Enter playing time (minutes):

Displaying Details

********** BOOK DETAILS **********
Title : C++ Primer
Price : 450.5
Page Count : 800

********** DISC DETAILS **********
Title : Mastering Audio
Price : 299.99
Playing Time : 60 minutes
```

EXP 4

EXP4.cpp

Matrix Operations (Addition, Subtraction & Multiplication)

Aim / Objective: Design a 'Matrix' class to perform standard linear algebra operations including addition, subtraction, and multiplication on two-dimensional matrices with row/column compatibility checks and validation.
Core Concepts: 2D Arrays in Classes, Object as Function Arguments, Return by Value, Matrix Algebra

SOURCE CODE:

```
#include <iostream>
using namespace std;

class Matrix
{
private:
    int row, col;
    int a[10][10];

public:
    void readMatrix()
    {
        cout << "Enter number of rows and columns: ";
        cin >> row >> col;

        cout << "Enter matrix elements:\n";
        for (int i = 0; i < row; i++)
        {
            for (int j = 0; j < col; j++)
            {
                cin >> a[i][j];
            }
        }
    }

    void displayMatrix()
    {
        for (int i = 0; i < row; i++)
        {
            for (int j = 0; j < col; j++)
            {
                cout << a[i][j] << "\t";
            }
            cout << endl;
        }
    }

    Matrix addMatrix(Matrix m)
    {
        Matrix temp;

        if (row != m.row || col != m.col)
        {
            cout << "Addition not possible.\n";
            temp.row = temp.col = 0;
            return temp;
        }

        temp.row = row;
        temp.col = col;
```

```
        for (int i = 0; i < row; i++)
        {
            for (int j = 0; j < col; j++)
            {
                temp.a[i][j] = a[i][j] + m.a[i][j];
            }
        }

        return temp;
    }

    Matrix subtractMatrix(Matrix m)
    {
        Matrix temp;

        if (row ≠ m.row || col ≠ m.col)
        {
            cout << "Subtraction not possible.\n";
            temp.row = temp.col = 0;
            return temp;
        }

        temp.row = row;
        temp.col = col;

        for (int i = 0; i < row; i++)
        {
            for (int j = 0; j < col; j++)
            {
                temp.a[i][j] = a[i][j] - m.a[i][j];
            }
        }

        return temp;
    }

    Matrix multiplyMatrix(Matrix m)
    {
        Matrix temp;

        if (col ≠ m.row)
        {
            cout << "Multiplication not possible.\n";
            temp.row = temp.col = 0;
            return temp;
        }

        temp.row = row;
        temp.col = m.col;

        for (int i = 0; i < temp.row; i++)
        {
            for (int j = 0; j < temp.col; j++)
            {
                temp.a[i][j] = 0;

                for (int k = 0; k < col; k++)
                {
                    temp.a[i][j] += a[i][k] * m.a[k][j];
                }
            }
        }

        return temp;
    }

    bool isValid()
```

```
    {
        return (row > 0 && col > 0);
    }
};

int main()
{
    Matrix A, B, C;

    cout << "Enter First Matrix\n";
    A.readMatrix();

    cout << "\nEnter Second Matrix\n";
    B.readMatrix();

    cout << "\nFirst Matrix:\n";
    A.displayMatrix();

    cout << "\nSecond Matrix:\n";
    B.displayMatrix();

    cout << "\nAddition of Matrices:\n";
    C = A.addMatrix(B);
    if (C.isValid())
        C.displayMatrix();

    cout << "\nSubtraction of Matrices:\n";
    C = A.subtractMatrix(B);
    if (C.isValid())
        C.displayMatrix();

    cout << "\nMultiplication of Matrices:\n";
    C = A.multiplyMatrix(B);
    if (C.isValid())
        C.displayMatrix();

    return 0;
}
```

TEST INPUT:

```
Matrix A (2x2): [[1, 2], [3, 4]]
Matrix B (2x2): [[5, 6], [7, 8]]
```

EXECUTION OUTPUT:

```
Terminal — ./a.out

Enter First Matrix
Enter number of rows and columns: Enter matrix elements:

Enter Second Matrix
Enter number of rows and columns: Enter matrix elements:

First Matrix:
1       2
3       4

Second Matrix:
5       6
7       8

Addition of Matrices:
```

```
6       8
10      12

Subtraction of Matrices:
-4      -4
-4      -4

Multiplication of Matrices:
19      22
43      50
```

EXP 5

EXP5.cpp

Multiple Inheritance, Constructors & Destructors Order

Aim / Objective: Demonstrate the order of execution of constructors and destructors in multiple inheritance where class 'Result' derives from base classes 'Student' and 'Sports'. Test default, parameterized, and copy constructors.
Core Concepts: Multiple Inheritance, Constructor Chaining, Copy Constructor, Destructor Invocation Order

SOURCE CODE:

```
#include<iostream>
using namespace std;
class Student {
protected:
    int rollNo;
public:
    Student() {
        rollNo = 0;
        cout << "Student Default Constructor Called" << endl;
    }
    Student(int r) {
        rollNo = r;
        cout << "Student Parameterized Constructor Called" << endl;
    }
    Student(const Student &s) {
        rollNo = s.rollNo;
        cout << "Student Copy Constructor Called" << endl;
    }
    ~Student() {
        cout << "Student Destructor Called" << endl;
    }
};

class Sports {
protected:
    int sportsMarks;
public:
    Sports() {
        sportsMarks = 0;
        cout << "Sports Default Constructor Called" << endl;
    }
    Sports(int m) {
        sportsMarks = m;
        cout << "Sports Parameterized Constructor Called" << endl;
    }
    Sports(const Sports &s) {
        sportsMarks = s.sportsMarks;
        cout << "Sports Copy Constructor Called" << endl;
    }
    ~Sports() {
        cout << "Sports Destructor Called" << endl;
    }
};


class Result : public Student, public Sports {
private:
    int totalMarks;
public:
```

```
    Result() : Student(), Sports() {
        totalMarks = 0;
        cout << "Result Default Constructor Called" << endl;
    }
    Result(int r, int s, int t) : Student(r), Sports(s) {
        totalMarks = t;
        cout << "Result Parameterized Constructor Called" << endl;
    }
    Result(const Result &obj) : Student(obj), Sports(obj) {
        totalMarks = obj.totalMarks;
        cout << "Result Copy Constructor Called" << endl;
    }
    void display() {
        cout << "\nRoll No : " << rollNo;
        cout << "\nSports Marks : " << sportsMarks;
        cout << "\nTotal Marks : " << totalMarks << endl;
    }
    ~Result() {
        cout << "Result Destructor Called" << endl;
    }
};


int main() {
    cout << "\n===== Default Constructor =====\n";
    Result r1;
    r1.display();

    cout << "\n===== Parameterized Constructor =====\n";
    Result r2(101, 25, 450);
    r2.display();

    cout << "\n===== Copy Constructor =====\n";
    Result r3(r2);
    r3.display();

    cout << "\nProgram Ends..." << endl;
    return 0;
}
```

TEST INPUT:

> None (Demonstrated programmatically in main)

EXECUTION OUTPUT:

Terminal — ./a.out

```
===== Default Constructor =====
Student Default Constructor Called
Sports Default Constructor Called
Result Default Constructor Called

Roll No : 0
Sports Marks : 0
Total Marks : 0

===== Parameterized Constructor =====
Student Parameterized Constructor Called
Sports Parameterized Constructor Called
Result Parameterized Constructor Called

Roll No : 101
```

```
Sports Marks : 25
Total Marks : 450

===== Copy Constructor =====
Student Copy Constructor Called
Sports Copy Constructor Called
Result Copy Constructor Called

Roll No : 101
Sports Marks : 25
Total Marks : 450

Program Ends...
Result Destructor Called
Sports Destructor Called
Student Destructor Called
Result Destructor Called
Sports Destructor Called
Student Destructor Called
Result Destructor Called
Sports Destructor Called
Student Destructor Called
```

EXP 6

EXP6.cpp

File Input and Output Operations (ofstream & ifstream)

Aim / Objective: Write a C++ program that creates an output file stream, writes formatted data (strings and numeric values) to a text file, safely closes it, and then opens an input file stream to read and display file contents.
Core Concepts: File Streams, Persistent Storage, Error Handling with is_open(), Line-by-line reading

SOURCE CODE:

```
#include <iostream>
#include <fstream>
#include <string>

int main() {
    const std::string filename = "sample_output.txt";

    std::ofstream outFile(filename);

    if (!outFile.is_open()) {
        std::cerr << "Error: Could not create or open the output file." << std::endl;
        return 1;
    }

    std::cout << "Writing data to '" << filename << "'...\n";
    outFile << "Line 1: Hello, C++ File I/O!\n";
    outFile << "Line 2: Demonstration of writing to a file.\n";
    outFile << "Line 3: Storing integer value = " << 100 << "\n";
    outFile << "Line 4: NMIET " << "\n";


    outFile.close();
    std::cout << "File closed successfully after writing.\n\n";

    std::ifstream inFile(filename);

    if (!inFile.is_open()) {
        std::cerr << "Error: Could not open the file for reading." << std::endl;
        return 1;
    }

    std::cout << "--- Contents of '" << filename << "' ---" << std::endl;

    std::string line;
    while (std::getline(inFile, line)) {
        std::cout << line << std::endl;
    }

    inFile.close();
    std::cout << "-------------------------------------------------" << std::endl;

    return 0;
}
```

TEST INPUT:

None (Data written and read from 'sample_output.txt')

EXECUTION OUTPUT:

```
Terminal — ./a.out

Writing data to 'sample_output.txt' ...
File closed successfully after writing.

--- Contents of 'sample_output.txt' ---
Line 1: Hello, C++ File I/O!
Line 2: Demonstration of writing to a file.
Line 3: Storing integer value = 100
Line 4: NMIET
------------------------------------------------
```

EXP 7

EXP7.cpp

Demonstration of All Types of Inheritance

Aim / Objective: Implement and demonstrate the four fundamental types of inheritance in C++: Single Inheritance, Multiple Inheritance, Multilevel Inheritance, and Hierarchical Inheritance using clean class hierarchies.
Core Concepts: Single, Multiple, Multilevel, and Hierarchical Inheritance, Namespaces, Code Organization

SOURCE CODE:

```
// Experiment 7: Demonstration of Types of Inheritance
// 1. Single Inheritance
// 2. Multiple Inheritance
// 3. Multilevel Inheritance
// 4. Hierarchical Inheritance

#include <iostream>
using namespace std;

// 1. Single Inheritance (7-1- single)
namespace SingleInheritance {
class Student {
public:
  void getStudent() { cout << "Student details" << endl; }
};

class Result : public Student {
public:
  void displayResult() { cout << "Result: Pass" << endl; }
};

void run() {
  Result r;
  r.getStudent();    // Base class function
  r.displayResult(); // Derived class function
}
} // namespace SingleInheritance

// 2. Multiple Inheritance (7-2- multiple)
namespace MultipleInheritance {
class Academic {
public:
  void academicMarks() { cout << "Academic Marks: 80" << endl; }
};

class Sports {
public:
  void sportsMarks() { cout << "Sports Marks: 15" << endl; }
};

class Student : public Academic, public Sports {
public:
  void display() { cout << "Total Performance: Good" << endl; }
};

void run() {
  Student s;
  s.academicMarks();
  s.sportsMarks();
  s.display();
```

```
}
} // namespace MultipleInheritance

// 3. Multilevel Inheritance (7-3- multilevel)
namespace MultilevelInheritance {
class Person {
public:
  void showPerson() { cout << "I am a Person" << endl; }
};

class Student : public Person {
public:
  void showStudent() { cout << "I am a Student" << endl; }
};

class Result : public Student {
public:
  void showResult() { cout << "Result: Pass" << endl; }
};

void run() {
  Result r;
  r.showPerson();  // Person class
  r.showStudent(); // Student class
  r.showResult();  // Result class
}
} // namespace MultilevelInheritance

// 4. Hierarchical Inheritance (7-4- hierarchical)
namespace HierarchicalInheritance {
class Employee {
public:
  void showEmployee() { cout << "Employee ID: 101" << endl; }
};

class Developer : public Employee {
public:
  void showDeveloper() { cout << "Role: Software Developer" << endl; }
};

class Manager : public Employee {
public:
  void showManager() { cout << "Role: Project Manager" << endl; }
};

void run() {
  Developer d;
  Manager m;

  cout << "Developer Details:" << endl;
  d.showEmployee();
  d.showDeveloper();

  cout << "\nManager Details:" << endl;
  m.showEmployee();
  m.showManager();
}
} // namespace HierarchicalInheritance

int main() {
  cout << "=== 1. Single Inheritance ===" << endl;
  SingleInheritance::run();

  cout << "\n=== 2. Multiple Inheritance ===" << endl;
  MultipleInheritance::run();

  cout << "\n=== 3. Multilevel Inheritance ===" << endl;
  MultilevelInheritance::run();
```

```
  cout << "\n=== 4. Hierarchical Inheritance ===" << endl;
  HierarchicalInheritance::run();

  return 0;
}
```

TEST INPUT:

> None (Demonstrated programmatically in main)

EXECUTION OUTPUT:

Terminal — ./a.out

```
=== 1. Single Inheritance ===
Student details
Result: Pass

=== 2. Multiple Inheritance ===
Academic Marks: 80
Sports Marks: 15
Total Performance: Good

=== 3. Multilevel Inheritance ===
I am a Person
I am a Student
Result: Pass

=== 4. Hierarchical Inheritance ===
Developer Details:
Employee ID: 101
Role: Software Developer

Manager Details:
Employee ID: 101
Role: Project Manager
```

EXP 8

EXP8.cpp

Generic Class Template (Library Management System)

Aim / Objective: Implement a generic Library Management System using C++ Class Templates where Book ID can be instantiated with any datatype (e.g. integer for numeric cataloging, string for alphanumeric ISBN codes).
Core Concepts: Class Templates, Generic Programming, Type Independence, Encapsulation

SOURCE CODE:

```
#include <iostream>
#include <string>

using namespace std;

// Class Template
template <class T> class Library {
private:
  T bookID;
  string bookName;
  string author;
  bool issued;

public:
  // Constructor
  Library(T id, string name, string auth) {
    bookID = id;
    bookName = name;
    author = auth;
    issued = false;
  }

  // Display book details
  void display() {
    cout << "\nBook ID: " << bookID;
    cout << "\nBook Name: " << bookName;
    cout << "\nAuthor: " << author;
    cout << "\nStatus: ";
    if (issued)
      cout << "Issued";
    else
      cout << "Available";
    cout << "\n";
  }

  // Issue book
  void issueBook() {
    if (!issued) {
      issued = true;
      cout << "\nBook issued successfully.\n";
    } else {
      cout << "\nBook is already issued.\n";
    }
  }

  // Return book
  void returnBook() {
    if (issued) {
      issued = false;
      cout << "\nBook returned successfully.\n";
```

```
    } else {
      cout << "\nBook was not issued.\n";
    }
  }
};

int main() {
  // Library object with integer Book ID
  Library<int> book1(101, "Object Oriented Programming", "Robert Lafore");

  // Library object with string Book ID
  Library<string> book2("CS102", "Database Management System", "Korth");

  cout << "LIBRARY MANAGEMENT SYSTEM\n";

  cout << "\nBook 1 Details:";
  book1.display();

  cout << "\nIssuing Book 1:";
  book1.issueBook();

  cout << "\nBook 1 Details after issuing:";
  book1.display();

  cout << "\n————————————————————————————\n";

  cout << "\nBook 2 Details:";
  book2.display();

  return 0;
}
```

TEST INPUT:

> None (Demonstrated programmatically in main)

EXECUTION OUTPUT:

Terminal — ./a.out

```
LIBRARY MANAGEMENT SYSTEM

Book 1 Details:
Book ID: 101
Book Name: Object Oriented Programming
Author: Robert Lafore
Status: Available

Issuing Book 1:
Book issued successfully.

Book 1 Details after issuing:
Book ID: 101
Book Name: Object Oriented Programming
Author: Robert Lafore
Status: Issued

————————————————————————————

Book 2 Details:
Book ID: CS102
Book Name: Database Management System
```

```
Author: Korth
Status: Available
```

EXP 9

EXP9.cpp

Standard Template Library (STL Map for State Population)

Aim / Objective: Write a C++ program using the Standard Template Library (STL) map container that stores state names and their respective populations. Prompt the user for a state name and retrieve its population using map iterators.
Core Concepts: STL Containers, std::map, Key-Value Pairs, Iterators, find() Method

SOURCE CODE:

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
  map<string, long long> statePopulation;

  statePopulation["Maharashtra"] = 126000000;
  statePopulation["Gujarat"] = 72000000;
  statePopulation["Karnataka"] = 68000000;
  statePopulation["Rajasthan"] = 82000000;
  statePopulation["Tamil Nadu"] = 77000000;
  statePopulation["Uttar Pradesh"] = 240000000;
  statePopulation["Madhya Pradesh"] = 88000000;

  string stateName;

  cout << "Enter the name of a state: ";
  getline(cin, stateName);

  map<string, long long>::iterator it = statePopulation.find(stateName);

  if (it != statePopulation.end()) {
    cout << "Population of " << stateName << " = " << it->second << endl;
  } else {
    cout << "State not found in the map." << endl;
  }

  return 0;
}
```

TEST INPUT:

```
Enter the name of a state: Maharashtra
```

EXECUTION OUTPUT:

```
Terminal — ./a.out

Enter the name of a state: Population of Maharashtra = 126000000
```

EXP 10

EXP10.cpp

ATM Machine Simulation & Custom Exception Handling

Aim / Objective: Create a robust ATM Machine simulation using C++ object-oriented principles. Handle exceptions for invalid transactions (negative deposit/withdrawal, insufficient funds, non-numeric input) with appropriate custom exception classes.
Core Concepts: Exception Hierarchy, Custom Exceptions (Arithmetic & IllegalArgument), Input Validation, Menu-driven loop

SOURCE CODE:

```
#include <iostream>
#include <limits>
#include <stdexcept>

using namespace std;

// Custom exception for insufficient funds
class ArithmeticException : public runtime_error {
public:
  ArithmeticException(const string &message) : runtime_error(message) {}
};

// Custom exception for invalid input
class IllegalArgumentException : public invalid_argument {
public:
  IllegalArgumentException(const string &message) : invalid_argument(message) {}
};

class ATM {
private:
  double balance;

public:
  ATM(double initialBalance) { balance = initialBalance; }

  void checkBalance() {
    cout << "\nCurrent Account Balance: Rs. " << balance << endl;
  }

  void deposit(double amount) {
    if (amount ≤ 0) {
      throw IllegalArgumentException(
          "Deposit amount must be greater than zero.");
    }

    balance += amount;

    cout << "Rs. " << amount << " deposited successfully.\n";
    cout << "Updated Balance: Rs. " << balance << endl;
  }

  void withdraw(double amount) {
    if (amount ≤ 0) {
      throw IllegalArgumentException(
          "Withdrawal amount must be greater than zero.");
    }
```

```
    if (amount > balance) {
      throw ArithmeticException("Insufficient funds.");
    }

    balance -= amount;

    cout << "Rs. " << amount << " withdrawn successfully.\n";
    cout << "Remaining Balance: Rs. " << balance << endl;
  }
};

int main() {
  ATM atm(10000);
  int choice;
  double amount;

  cout << "=====================================\n";
  cout << "        ATM MACHINE SIMULATION      \n";
  cout << "=====================================\n";

  while (true) {
    cout << "\n---------- ATM MENU ----------\n";
    cout << "1. Check Account Balance\n";
    cout << "2. Deposit Money\n";
    cout << "3. Withdraw Money\n";
    cout << "4. Exit\n";
    cout << "-------------------------------\n";
    cout << "Enter your choice: ";

    try {
      cin >> choice;

      if (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');

        throw IllegalArgumentException(
            "Invalid input. Enter a number from 1 to 4.");
      }

      switch (choice) {
      case 1:
        atm.checkBalance();
        break;

      case 2:
        cout << "Enter amount to deposit: ";
        cin >> amount;

        if (cin.fail()) {
          cin.clear();
          cin.ignore(numeric_limits<streamsize>::max(), '\n');

          throw IllegalArgumentException("Invalid deposit amount.");
        }

        atm.deposit(amount);
        break;

      case 3:
        cout << "Enter amount to withdraw: ";
        cin >> amount;

        if (cin.fail()) {
          cin.clear();
          cin.ignore(numeric_limits<streamsize>::max(), '\n');

          throw IllegalArgumentException("Invalid withdrawal amount.");
```

```
        }

        atm.withdraw(amount);
        break;

      case 4:
        cout << "\nThank you for using the ATM.\n";
        return 0;

      default:
        throw IllegalArgumentException("Invalid choice. Please select 1 to 4.");
      }
    } catch (const ArithmeticException &e) {
      cout << "\nArithmetic Exception: " << e.what() << endl;
    } catch (const IllegalArgumentException &e) {
      cout << "\nIllegal Argument Exception: " << e.what() << endl;
    } catch (const exception &e) {
      cout << "\nException: " << e.what() << endl;
    }

    // Finally equivalent in C++
    cout << "\nTransaction completed.\n";
    cout << "ATM is ready for the next transaction.\n";
  }

  return 0;
}
```

TEST INPUT:

> 1 (Check Balance) -> 2 (Deposit 5000) -> 3 (Withdraw 3000) -> 4 (Exit)

EXECUTION OUTPUT:

```
Terminal — ./a.out

═══════════════════════════════════
        ATM MACHINE SIMULATION
═══════════════════════════════════

──────────── ATM MENU ────────────
1. Check Account Balance
2. Deposit Money
3. Withdraw Money
4. Exit
──────────────────────────────────
Enter your choice:
Current Account Balance: Rs. 10000

Transaction completed.
ATM is ready for the next transaction.

──────────── ATM MENU ────────────
1. Check Account Balance
2. Deposit Money
3. Withdraw Money
4. Exit
──────────────────────────────────
Enter your choice: Enter amount to deposit: Rs. 5000 deposited successfully.
Updated Balance: Rs. 15000

Transaction completed.
```

```
ATM is ready for the next transaction.

──────────── ATM MENU ────────────
1. Check Account Balance
2. Deposit Money
3. Withdraw Money
4. Exit
──────────────────────────────────
Enter your choice: Enter amount to withdraw: Rs. 3000 withdrawn successfully.
Remaining Balance: Rs. 12000

Transaction completed.
ATM is ready for the next transaction.

──────────── ATM MENU ────────────
1. Check Account Balance
2. Deposit Money
3. Withdraw Money
4. Exit
──────────────────────────────────
Enter your choice:
Thank you for using the ATM.
```